

AI/ML Internship – Day 71 Notes & Tasks

Module 9: Building a PDF Question Answering Chatbot using LangChain & ChromaDB

Part 1: Recap of Day 70

Yesterday we learned:

-  Complete RAG Pipeline
-  Document Upload
-  Chunking
-  Embeddings
-  Vector Database
-  Response Generation

Today we will build a complete application that allows users to upload a PDF and ask questions about it.

Part 2: What is a PDF Question Answering Chatbot?

Definition

A **PDF Question Answering Chatbot** is an AI application that allows users to upload PDF documents and ask questions in natural language.

The chatbot reads the PDF, retrieves the relevant information, and generates an accurate answer.

Simple Definition

Instead of reading a 100-page PDF manually, the chatbot finds the answer for you.

Part 3: Real-Life Example

Imagine a college uploads its admission brochure.

Student asks:

What is the admission fee?

The chatbot searches the brochure and replies:

The admission fee is ₹25,000.

Another question:

Who is eligible for admission?

↓

The chatbot searches the PDF and gives the correct answer.

Part 4: Applications

PDF Question Answering is used in:

- Company HR Manuals
 - Banking Documents
 - Insurance Policies
 - Medical Guidelines
 - College Brochures
 - Research Papers
 - Government Rules
 - Product Manuals
-

Part 5: Architecture

Upload PDF

|



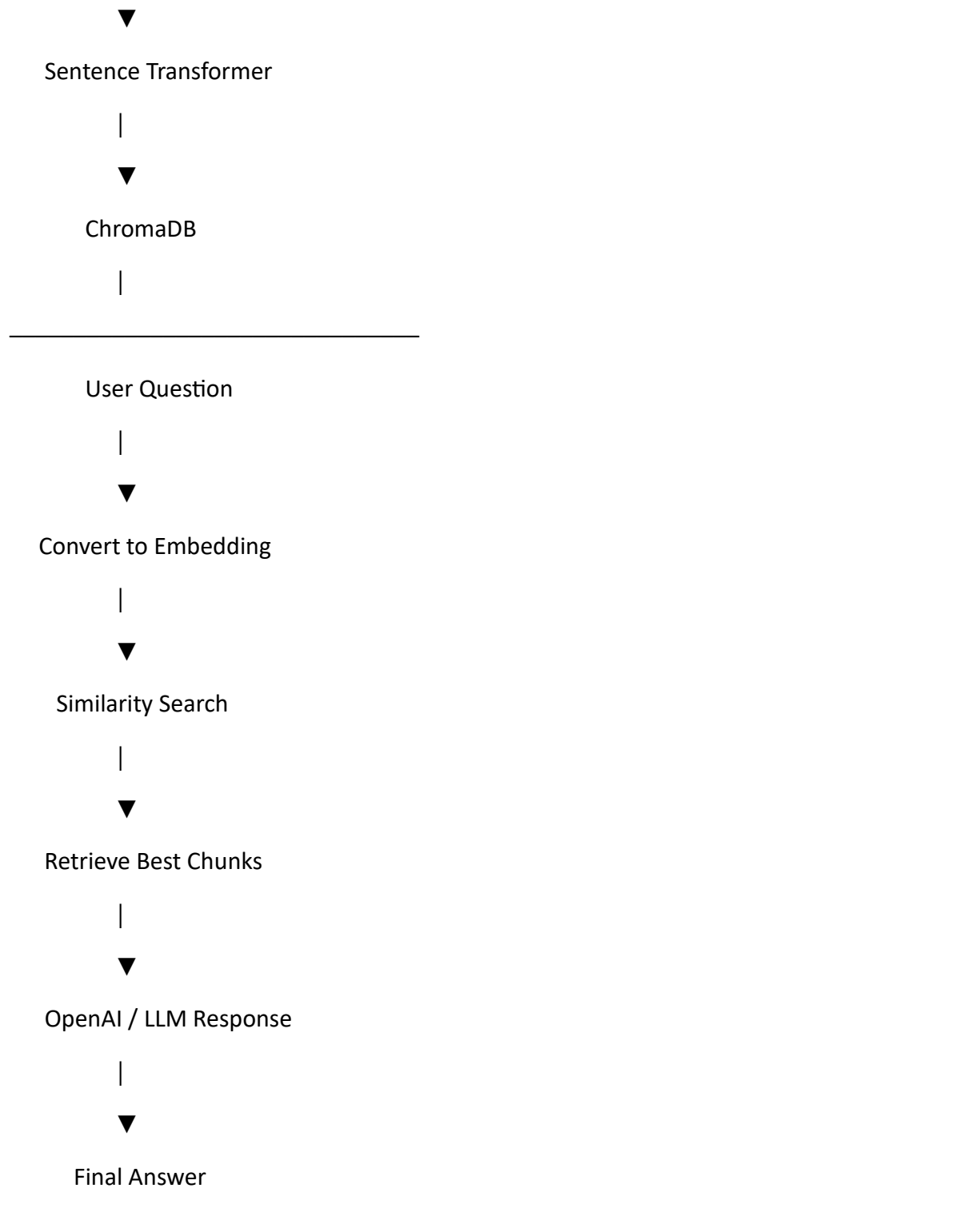
PDF Text Extraction

|



Text Chunking

|



📌 Part 6: Required Libraries

Install the following packages:

```
pip install langchain
```

```
pip install langchain-community
```

```
pip install langchain-openai
```

```
pip install sentence-transformers
```

```
pip install chromadb
```

```
pip install pypdf
```

```
pip install python-dotenv
```

Part 7: Step 1 – Load PDF

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("college.pdf")
```

```
documents = loader.load()
```

The PDF is converted into pages that can be processed by LangChain.

Part 8: Step 2 – Split the Text

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50  
)
```

```
chunks = splitter.split_documents(documents)
```

Why chunk overlap?

Because important information may continue across two chunks. Overlap helps preserve context.

Part 9: Step 3 – Create Embeddings

```
from langchain.embeddings import HuggingFaceEmbeddings
```

```
embedding = HuggingFaceEmbeddings(  
    model_name="all-MiniLM-L6-v2"  
)
```

Each chunk is converted into an embedding.

Part 10: Step 4 – Store in ChromaDB

```
from langchain.vectorstores import Chroma
```

```
db = Chroma.from_documents(  
    chunks,  
    embedding  
)
```

The vector database stores all document embeddings.

Part 11: Step 5 – User Question

Example:

What is the internship duration?

This question is converted into an embedding.

Part 12: Step 6 – Retrieve Similar Chunks

```
docs = db.similarity_search(  
    "What is the internship duration?",  
    k=3  
)
```

The top 3 most relevant chunks are retrieved.

Part 13: Step 7 – Generate the Final Answer

The retrieved chunks are sent to the Large Language Model.

Example:

Context:

Internship Duration is 3 Months.

Question:

How long is the internship?

AI Response:

The internship duration is 3 months.

Part 14: Complete Workflow

Upload PDF

|



Load PDF

|



Split into Chunks

|



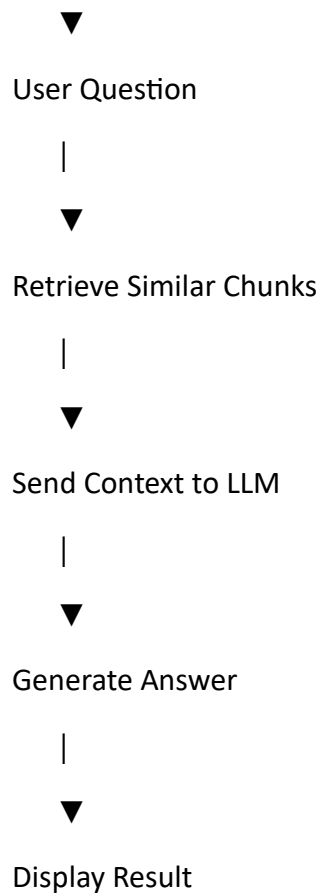
Generate Embeddings

|



Store in ChromaDB

|



📌 Part 15: Why LangChain?

LangChain simplifies AI application development.

It provides ready-made tools for:

- PDF loading
- Chunking
- Embeddings
- Vector databases
- Prompt management
- Chat memory
- LLM integration

Instead of writing hundreds of lines of code, developers can build AI applications with much less effort.

Part 16: Advantages

- ✓ Reads large PDF files
 - ✓ Fast semantic search
 - ✓ Accurate responses
 - ✓ Easy to integrate with OpenAI
 - ✓ Supports multiple documents
-

Part 17: Limitations

- ✗ Poor quality PDFs reduce accuracy
 - ✗ Scanned PDFs require OCR
 - ✗ Large documents require more storage
 - ✗ API costs may apply when using cloud LLMs
-

Part 18: Theory Questions

What is a PDF Question Answering Chatbot?

A chatbot that allows users to upload PDF documents and ask questions. It retrieves relevant information from the document and generates answers using AI.

Why do we use LangChain?

LangChain simplifies the development of AI applications by providing components for document loading, chunking, embeddings, retrieval, and LLM integration.

Why do we use ChromaDB?

ChromaDB stores document embeddings and enables fast semantic retrieval.

What is similarity search?

Similarity search finds document chunks whose meanings are closest to the user's question.

Why are embeddings important?

Embeddings convert text into numerical vectors that allow semantic comparison.

Part 19: Interview Questions

Explain the workflow of a PDF Question Answering Chatbot.

The workflow includes loading the PDF, splitting it into chunks, creating embeddings, storing them in a vector database, retrieving relevant chunks using similarity search, and generating the final answer using an LLM.

Why is chunk overlap used?

Chunk overlap preserves context when information spans multiple chunks, improving retrieval quality.

What is LangChain?

LangChain is a framework for developing AI-powered applications using LLMs, document loaders, vector databases, and retrieval pipelines.

Name three libraries used in a RAG chatbot.

- LangChain
 - ChromaDB
 - Sentence Transformers
-

What is the purpose of ChromaDB?

To store embeddings and retrieve relevant document chunks efficiently.

Tasks for Day 71

Theory

1. Define a PDF Question Answering Chatbot.
2. Explain the complete workflow.

3. What is LangChain?
 4. Why do we use ChromaDB?
 5. Explain similarity search.
-

Practical Task 1

Install all required libraries:

```
pip install langchain
```

```
pip install langchain-community
```

```
pip install chromadb
```

```
pip install sentence-transformers
```

```
pip install pypdf
```

Practical Task 2

Load a sample PDF using PyPDFLoader.

Practical Task 3

Split the PDF into chunks using RecursiveCharacterTextSplitter.

Practical Task 4

Store the chunks in ChromaDB and perform a similarity search for three different questions.

Practical Task 5

Draw the complete architecture of the PDF Question Answering Chatbot.